

Descripción

El objetivo de este proyecto es desmitificar el funcionamiento de las Redes Neuronales Artificiales (ANN) construyendo una desde cero, sin el uso de librerías de Machine Learning. El estudiante deberá programar la arquitectura matemática que soporta el "Forward Propagation" y "Backpropagation" (Gradiente Descendente) aplicando "Matemáticas Transparentes".

El proyecto consiste en una aplicación web interactiva desarrollada en Python que sea capaz de capturar un número (0-9) escrito a mano mediante la cámara del dispositivo, procesar la imagen y predecir qué número es utilizando la red neuronal construida. Además, el sistema funcionará como una herramienta educativa visual, mostrando en tiempo real los cálculos, la actualización de pesos y el flujo de los datos a través de los nodos de la red.

Objetivos Generales

- Construir una Red Neuronal Artificial desde cero para la clasificación multiclase (Reconocimiento de Dígitos), evidenciando los cálculos de álgebra lineal y cálculo diferencial involucrados.
- Desarrollar una interfaz web interactiva que permita la captura de imágenes en tiempo real y la visualización del proceso de aprendizaje de la máquina.

Objetivos Específicos

- Comprender e implementar matemáticamente el proceso de propagación hacia adelante (*Forward Propagation*) y propagación hacia atrás (*Backpropagation*).
- Visualizar de manera interactiva la topología de la red neuronal, los pesos sinápticos y los errores.
- Aplicar procesamiento de imágenes básico para adaptar capturas de cámara web al formato requerido por la red neuronal.

Requisitos del Proyecto

1. Arquitectura de la Red Neuronal (Matemáticas Transparentes)

Se debe construir una red neuronal del tipo Perceptrón Multicapa (MLP) con la siguiente topología estricta:

- **Capa de Entrada (Input Layer):** 784 nodos (correspondientes a los 784 píxeles de una imagen en escala de grises de 28x28 píxeles).
- **Capa Oculta (Hidden Layer):** 64 nodos. El estudiante debe implementar matemáticamente la función de activación (ej. Sigmoide o ReLU).
- **Capa de Salida (Output Layer):** 10 nodos (representando las probabilidades de los dígitos del 0 al 9). Se debe implementar la función Softmax para la clasificación final.

Restricción Estricta: Queda **PROHIBIDO** el uso de librerías como TensorFlow, PyTorch, Keras, Scikit-Learn o similares. Solo se permite el uso de librerías matemáticas base como math o NumPy para operaciones de matrices.

2. Cálculos Manuales Obligatorios

Antes de pasar a la programación, el estudiante debe interiorizar las matemáticas del modelo:

- **Cálculo Manual de un Nodo:** El estudiante debe documentar (en un PDF o Excel) el recorrido matemático exacto de **una sola imagen** pasando por la red neuronal no entrenada. Debe mostrar el producto punto de la capa de entrada con los pesos, la aplicación de la función de activación en un nodo de la capa oculta, el cálculo del error y un paso de actualización de peso (Derivada / Backpropagation).
- El código final debe ser capaz de imprimir un modo "debug" para una sola iteración, demostrando que arroja **exactamente el mismo resultado** que su cálculo manual (con una precisión de al menos 4 decimales).

3. Captura y Procesamiento con Cámara Web

- La aplicación web debe activar la cámara del dispositivo del usuario.
- El usuario dibujará un dígito en una hoja de papel o pizarra y lo mostrará a la cámara.

- El sistema debe capturar la imagen, convertirla a escala de grises, redimensionarla a 28x28 píxeles, aplanar la matriz a un vector de 784 posiciones y dársela como entrada a la Red Neuronal para su predicción.

4. Interfaz Visual e Interactividad en Tiempo Real

La aplicación web (que debe estar creada en Python) debe tener una sección de entrenamiento y visualización con las siguientes características:

- **Representación Gráfica de la Red:** Visualización de los nodos (neuronas) y sus conexiones. Las conexiones (pesos) deben cambiar visualmente (grosor o color) según su valor numérico para ver cómo interactúan.
- **Control de Velocidad:** El proceso de entrenamiento/predicción debe tener al menos dos modos:
 - *Ejecución Rápida:* Entrena la red a máxima velocidad para llegar a un modelo funcional.
 - *Cámara Lenta (Modo Educativo):* Permite ver paso a paso cómo se propagan los datos de la capa de entrada a la salida, y cómo los errores "regresan" para actualizar los pesos. con botones de siguiente paso.
- **Estadísticas en Tiempo Real:** Un panel o *dashboard* que muestre en vivo:
 - Época (Epoch) actual y número de iteración.
 - Valor del Error / Función de Pérdida (Loss).
 - Precisión (Accuracy) actual.

5. Reportes y Bitácoras a Generar

El sistema debe generar obligatoriamente los siguientes archivos de control:

- **Bitácora de Matrices:** Un archivo de texto (.txt o .log) generado por el código durante el entrenamiento, donde se imprima una sección representativa de la matriz de pesos de la capa oculta exactamente en las iteraciones: 1, 50 y 100. Esto evidenciará el cambio matemático.

- **Gráfica de la Función de Costo:** La interfaz web debe graficar en tiempo real cómo la curva de error desciende suavemente conforme avanzan las épocas de entrenamiento. Esta gráfica debe poder exportarse o visualizarse claramente al finalizar.

Consideraciones Técnicas

- **Lenguaje:** Python.

Entregables

La entrega se realizará por medio de la plataforma Classroom. En el repositorio de Git se debe incluir:

1. Código fuente completo (Frontend, Backend y Motor de Red Neuronal desde cero).
2. Documento de **Cálculo Manual** (PDF o Excel).
3. Archivos de reporte generados por el sistema (Bitácora de Matrices).
4. Manual Técnico (explicando las derivadas parciales y álgebra utilizada).
5. Manual de Usuario (Instrucciones para probar con la cámara).

Importante

- Copias obtendrán nota de cero y se notificará a coordinación.
- Se validará estrictamente en la calificación que las matemáticas del cálculo manual cuadren con los resultados del programa en su primera iteración.